

ОГЛАВЛЕНИЕ

Введение.....	2
Постановка задачи.....	2
1 Реализация программного решения.....	4
1.1 Используемые средства разработки.....	4
1.2 Архитектура программы.....	4
1.3 Алгоритм обработки изображений.....	5
1.4 История разработки по коммитам.....	6
2 Оценка качества работы.....	7
Вывод.....	9
Список использованных источников.....	10
Приложение А. Листинг программы.....	11

ВВЕДЕНИЕ

Предварительная обработка изображений является одним из начальных этапов систем компьютерного зрения. Даже если дальнейший алгоритм предназначен для распознавания объектов, текста или границ, результат его работы зависит от качества входного изображения. Наличие шума, слабого контраста и лишних деталей может усложнить анализ и привести к ошибкам.

В данной работе исследуются базовые операции обработки изображений: преобразование цветовых пространств, сглаживание, повышение резкости, морфологические операции и фильтрация искусственно добавленного шума. Практическая часть позволяет не только увидеть результат каждого метода, но и определить, для какой задачи он подходит лучше.

Цель работы — разработать программу на языке Python, позволяющую выполнить и визуально сравнить основные методы предобработки изображений с применением библиотеки OpenCV.

ПОСТАНОВКА ЗАДАЧИ

Необходимо реализовать программный модуль, который получает набор из трёх изображений различного типа и формирует результаты их обработки для последующего сравнения.

Для выполнения работы требуется реализовать следующие функции:

- загрузка и отображение трёх изображений: портрета человека, городского пейзажа и изображения с текстом;
- корректное преобразование каналов BGR в RGB при выводе средствами Matplotlib;
- преобразование изображений в RGB, Grayscale и HSV с отображением отдельных каналов HSV;

- применение GaussianBlur, MedianBlur и AverageBlur с размерами ядра 5x5, 11x11 и 21x21;
- повышение резкости изображения с использованием ядра свёртки 3x3;
- бинаризация изображения с текстом и выполнение операций erosion, dilation, opening и closing;
- добавление гауссова шума и шума «соль-перец»;
- сравнение эффективности сглаживающих фильтров при удалении разных типов шума;
- формирование итоговой сравнительной таблицы и сохранение результатов в PNG.

Результатом работы программы являются изображения-сетки, сохранённые в папке outputs и пригодные для включения в отчёт.

1 РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ

1.1 Используемые средства разработки

Программа реализована на языке Python. Для загрузки изображений, преобразования цветовых пространств, выполнения фильтрации, бинаризации и морфологических операций используется библиотека OpenCV [1]. Работа с массивами пикселей и генерация случайного шума выполняются средствами NumPy [2]. Вывод сравнительных сеток и сохранение результатов в графические файлы реализованы с применением Matplotlib [3].

OpenCV загружает цветные изображения в формате BGR, тогда как Matplotlib ожидает представление RGB. Поэтому перед отображением выполняется преобразование `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`. Это позволяет сохранить естественные цвета исходных изображений.

В работе также используется HSV-представление. В отличие от RGB, в котором цвет формируется сочетанием трёх компонент, HSV разделяет оттенок, насыщенность и яркость. Поэтому выделение объекта по цвету удобнее выполнять по каналу H: изменение освещения в большей степени отражается на канале V, а оттенок объекта сохраняется более стабильным.

1.2 Архитектура программы

Финальная версия программы реализована в объектно-ориентированном стиле. Каждый класс выполняет отдельную группу операций, что упрощает понимание программы, позволяет независимо изменять алгоритмы фильтрации и отделяет обработку изображений от их визуализации.

Таблица 1 — Основные классы программы

Класс	Назначение	Основные методы
ImageItem	Хранение изображения	поля данных
ImageRepository	Загрузка файлов	load()
ColorSpaceProcessor	Цветовые преобразования	RGB, Gray, HSV
FilterProcessor	Фильтрация изображения	blur(), sharpen()
NoiseProcessor	Добавление шума	Gaussian, salt-pepper
MorphologyProcessor	Морфология маски	erode(), opening()

Visualizer	Вывод результатов	show_grid()
PreprocessingExperiment	Запуск сценария	run()

Класс `PreprocessingExperiment` объединяет все операции в единый сценарий: загружает изображения, запускает визуализацию цветовых пространств, проводит эксперименты с фильтрами, обрабатывает шумы и формирует итоговую сетку. Благодаря этому в функции `main()` остаётся только перечень путей к исходным файлам и запуск эксперимента.

1.3 Алгоритм обработки изображений

На первом этапе программа загружает три изображения из каталога `images`. Для визуальной проверки исходные файлы выводятся в одной сетке, а цветные изображения предварительно преобразуются из BGR в RGB.

Для каждого изображения формируются представления RGB и Grayscale, а также три отдельных канала HSV: H, S и V. Представление Grayscale удобно для операций, не требующих цвета, например бинаризации. HSV позволяет отдельно анализировать оттенок и яркость изображения.

Эксперимент со сглаживанием проводится на одном цветном изображении. Для каждого из трёх фильтров применяются ядра 5x5, 11x11 и 21x21. Увеличение ядра усиливает размытие: изображение становится менее детальным, но одновременно уменьшается влияние мелких шумов.

Повышение резкости выполняется оператором `cv2.filter2D` с ядром, у которого центральный коэффициент равен 5, а четыре соседних коэффициента равны -1. Такое преобразование подчёркивает границы и мелкие текстуры, но одновременно может сделать заметнее шум.

Морфологические операции выполняются над бинарным изображением, полученным пороговой бинаризацией фотографии с текстом. Эрозия уменьшает белые области, дилатация расширяет их, `opening` удаляет мелкие белые помехи, а `closing` закрывает небольшие чёрные разрывы внутри объектов.

На этапе исследования шума программа создаёт две зашумлённые версии изображения. Гауссов шум формируется добавлением случайных значений, распределённых по нормальному закону. Шум «соль-перец» создаётся заменой части пикселей на полностью белые и полностью чёрные. К обеим версиям применяются три сглаживающих фильтра для визуального сравнения.

1.4 История разработки по коммитам

Разработка программы выполнена в три последовательные итерации. Такой подход позволяет показать, как от базовой визуализации изображений решение постепенно расширяется до полноценного эксперимента с объектно-ориентированной архитектурой.

Таблица 2 — Итерации разработки программы

№	Коммит	Выполненные изменения
1	feat: add image loading and color space visualization	Загрузка изображений, преобразование BGR-RGB, вывод RGB, Grayscale и HSV.
2	feat: add smoothing, sharpening, morphology and noise experiments	Добавлены фильтры, резкость, морфология, генерация шума и сравнение очистки.
3	refactor: split preprocessing pipeline into classes and save final comparison grid	Введены классы, сохранение PNG и итоговая таблица результатов.

Первая итерация обеспечивает проверку корректности загрузки и отображения данных. Вторая итерация реализует все требуемые алгоритмические операции. Третья итерация приводит код к структуре, пригодной для сопровождения и повторного использования.

2 ОЦЕНКА КАЧЕСТВА РАБОТЫ

Качество работы программы оценивается не только по факту запуска, но и по корректности результата каждого этапа обработки. Для проверки используются три изображения разного типа, а для эксперимента с шумами исходное изображение выступает эталоном, с которым можно сопоставить результат фильтрации.

Таблица 3 — Критерии оценки качества работы программы

№	Критерий	Проверка	Успешный результат
1	Отображение	Просмотр рисунка 1	Цвета естественные, изображения не искажены
2	Цветовые пространства	Просмотр рисунка 2	Отображены RGB, Grayscale, H, S и V
3	Сглаживание	Просмотр рисунка 3	При росте ядра усиливается размытие
4	Резкость	Просмотр рисунка 4	Границы стали визуально чётче
5	Морфология	Просмотр рисунка 5	Видно расширение, сужение и очистку маски
6	Гауссов шум	Просмотр рисунка 6	GaussianBlur обеспечивает наиболее плавную очистку
7	Соль-перец	Просмотр рисунка 7	MedianBlur лучше удаляет точечные выбросы
8	Итоговая таблица	Просмотр рисунка 8	Представлены все три изображения и методы обработки

При искусственном добавлении шума качество фильтрации можно оценить количественно, так как исходное изображение известно. Для этого применяются показатели MSE и PSNR.

Среднеквадратическая ошибка определяется выражением: $MSE = (1 / N) \times \sum (I_i - K_i)^2$, где N — количество сравниваемых значений пикселей, I_i — значение пикселя исходного изображения, K_i — значение соответствующего

пикселя после фильтрации. Чем меньше MSE, тем ближе результат к исходному изображению.

Пиковое отношение сигнал/шум определяется выражением: $PSNR = 10 \times \lg(255^2 / MSE)$. Чем выше PSNR, тем меньше искажение изображения после удаления шума. Для реальных незашумлённых фотографий оценка выполняется визуально, поскольку эталонное изображение без искажений отсутствует.

После запуска в данный раздел необходимо вставить файлы, сформированные программой в каталоге outputs.

Таблица 4 — Сравнение исследованных методов обработки

Метод	Назначение	Наблюдаемый эффект
Grayscale	Упрощение данных до яркости	Цвет удаляется, сохраняются границы и контраст
HSV	Выделение информации о цвете	Оттенок отделён от яркости
GaussianBlur	Сглаживание и очистка гауссова шума	Мягкое размытие с сохранением общей структуры
MedianBlur	Удаление точечного шума	Хорошо убирает чёрные и белые выбросы
AverageBlur	Простое сглаживание	Размывает детали сильнее других методов
Sharpening	Усиление границ	Текстуры становятся чётче, но усиливается шум
Opening	Очистка белых помех в маске	Удаляются мелкие белые точки
Closing	Заполнение разрывов в маске	Закрываются небольшие чёрные отверстия

Ожидаемый результат эксперимента состоит в том, что гауссов шум наиболее эффективно сглаживается GaussianBlur, а шум «соль-перец» — MedianBlur. Это объясняется принципом работы фильтров: гауссов фильтр усредняет значения с весами, а медианный фильтр исключает влияние единичных резких выбросов.

ВЫВОД

В ходе работы разработана программа для исследования основных методов предобработки изображений средствами OpenCV. Реализованы загрузка и корректное отображение цветных изображений, преобразование в Grayscale и HSV, применение сглаживающих фильтров с различными ядрами, повышение резкости, бинаризация и морфологическая обработка.

Отдельно проведено исследование двух типов шума. Сравнение результатов показывает назначение фильтров: GaussianBlur целесообразно применять для подавления гауссова шума, а MedianBlur — для удаления точечных выбросов типа «соль-перец». Морфологические операции позволяют очищать бинарные маски от небольших дефектов.

Программа организована в виде набора классов, каждый из которых отвечает за отдельный этап обработки. Такое разделение делает код понятным и позволяет расширять эксперимент другими фильтрами, метриками качества или наборами изображений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OpenCV. Image Processing in OpenCV [Электронный ресурс]. URL: https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html (дата обращения: 28.05.2026).
2. NumPy. Random sampling: numpy.random.normal [Электронный ресурс]. URL: <https://numpy.org/doc/stable/reference/random/generated/numpy.random.normal.html> (дата обращения: 28.05.2026).
3. Matplotlib. Pyplot API: imshow, subplots, savefig [Электронный ресурс]. URL: https://matplotlib.org/stable/api/pyplot_summary.html (дата обращения: 28.05.2026).
4. OpenCV. Color Space Conversions [Электронный ресурс]. URL: https://docs.opencv.org/4.x/d8/d01/group_imgproc_color_conversions.html (дата обращения: 28.05.2026).
5. OpenCV. Morphological Transformations [Электронный ресурс]. URL: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html (дата обращения: 28.05.2026).

ПРИЛОЖЕНИЕ А. ЛИСТИНГ ПРОГРАММЫ

Ниже представлен основной листинг программы main.py.

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

from dataclasses import dataclass

@dataclass
class ImageItem:
    name: str
    path: str
    bgr: np.ndarray

class ImageRepository:
    def __init__(self, image_paths):
        self.image_paths = image_paths

    def load(self):
        images = []

        for name, path in self.image_paths.items():
            image = cv2.imread(path)

            if image is None:
                raise FileNotFoundError(f"Image not found: {path}")

            images.append(ImageItem(name=name, path=path, bgr=image))

        return images

class ColorSpaceProcessor:
    def to_rgb(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    def to_grayscale(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    def to_hsv(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    def get_hsv_channels(self, image):
        hsv = self.to_hsv(image)
        return cv2.split(hsv)

class FilterProcessor:
    def gaussian_blur(self, image, kernel_size):
        return cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)

    def median_blur(self, image, kernel_size):
        return cv2.medianBlur(image, kernel_size)

    def average_blur(self, image, kernel_size):
        return cv2.blur(image, (kernel_size, kernel_size))
```

```

def sharpen(self, image):
    kernel = np.array([
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0]
    ])

    return cv2.filter2D(image, -1, kernel)

def get_smoothing_results(self, image, kernel_sizes):
    results = []

    for size in kernel_sizes:
        results.append((f"Gaussian {size}x{size}", self.gaussian_blur(image, size)))
        results.append((f"Median {size}x{size}", self.median_blur(image, size)))
        results.append((f"Average {size}x{size}", self.average_blur(image, size)))

    return results

class NoiseProcessor:
    def add_gaussian_noise(self, image, mean=0, sigma=25):
        noise = np.random.normal(mean, sigma, image.shape)
        noisy = image.astype(np.float32) + noise
        return np.clip(noisy, 0, 255).astype(np.uint8)

    def add_salt_pepper_noise(self, image, amount=0.03):
        noisy = image.copy()
        total_pixels = image.shape[0] * image.shape[1]

        salt_count = int(total_pixels * amount / 2)
        pepper_count = int(total_pixels * amount / 2)

        salt_y = np.random.randint(0, image.shape[0], salt_count)
        salt_x = np.random.randint(0, image.shape[1], salt_count)

        pepper_y = np.random.randint(0, image.shape[0], pepper_count)
        pepper_x = np.random.randint(0, image.shape[1], pepper_count)

        noisy[salt_y, salt_x] = 255
        noisy[pepper_y, pepper_x] = 0

    return noisy

class MorphologyProcessor:
    def __init__(self, color_processor):
        self.color_processor = color_processor

    def binarize(self, image):
        gray = self.color_processor.to_grayscale(image)
        _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        return binary

    def erode(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.erode(binary, kernel, iterations=1)

    def dilate(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.dilate(binary, kernel, iterations=1)

    def opening(self, binary, kernel_size):

```

```

        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

    def closing(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    def get_morphology_results(self, image, kernel_sizes):
        binary = self.binarize(image)
        results = [("Binary", binary)]

        for size in kernel_sizes:
            results.append((f"Erosion {size}x{size}", self.erode(binary, size)))
            results.append((f"Dilation {size}x{size}", self.dilate(binary, size)))
            results.append((f"Opening {size}x{size}", self.opening(binary, size)))
            results.append((f"Closing {size}x{size}", self.closing(binary, size)))

        return results

class Visualizer:
    def __init__(self, color_processor, output_dir="outputs"):
        self.color_processor = color_processor
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)

    def prepare_image(self, image):
        if len(image.shape) == 2:
            return image, "gray"

        return self.color_processor.to_rgb(image), None

    def show_and_save_grid(self, title, items, rows, cols, filename, figsize=(16, 10)):
        plt.figure(figsize=figsize)
        plt.suptitle(title, fontsize=16)

        for index, (item_title, image) in enumerate(items, start=1):
            prepared_image, cmap = self.prepare_image(image)

            plt.subplot(rows, cols, index)
            plt.imshow(prepared_image, cmap=cmap)
            plt.title(item_title)
            plt.axis("off")

        plt.tight_layout()

        output_path = os.path.join(self.output_dir, filename)
        plt.savefig(output_path, dpi=200, bbox_inches="tight")
        plt.show()

    def show_originals(self, images):
        items = [(item.name, item.bgr) for item in images]
        self.show_and_save_grid(
            title="Original images",
            items=items,
            rows=1,
            cols=len(items),
            filename="01_originals.png",
            figsize=(15, 5)
        )

    def show_color_spaces(self, image_item):
        rgb = self.color_processor.to_rgb(image_item.bgr)
        grayscale = self.color_processor.to_grayscale(image_item.bgr)
        h, s, v = self.color_processor.get_hsv_channels(image_item.bgr)

```

```

        items = [
            ("RGB", cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)),
            ("Grayscale", grayscale),
            ("HSV Hue", h),
            ("HSV Saturation", s),
            ("HSV Value", v)
        ]

        self.show_and_save_grid(
            title=f"Color spaces: {image_item.name}",
            items=items,
            rows=1,
            cols=5,
            filename=f"02_color_spaces_{image_item.name.lower()}.png",
            figsize=(18, 4)
        )

    def show_smoothing(self, image_item, smoothing_results):
        self.show_and_save_grid(
            title=f"Smoothing filters: {image_item.name}",
            items=smoothing_results,
            rows=3,
            cols=3,
            filename="03_smoothing_filters.png",
            figsize=(15, 12)
        )

    def show_sharpening(self, image_item, sharpened):
        items = [
            ("Before", image_item.bgr),
            ("After", sharpened)
        ]

        self.show_and_save_grid(
            title=f"Sharpening: {image_item.name}",
            items=items,
            rows=1,
            cols=2,
            filename="04_sharpening.png",
            figsize=(12, 5)
        )

    def show_morphology(self, image_item, morphology_results):
        self.show_and_save_grid(
            title=f"Morphology: {image_item.name}",
            items=morphology_results,
            rows=4,
            cols=4,
            filename="05_morphology.png",
            figsize=(18, 16)
        )

    def show_noise_reduction(self, title, original, noisy, filtered_results, filename):
        items = [
            ("Original", original),
            ("Noisy", noisy)
        ]

        items.extend(filtered_results)

        self.show_and_save_grid(
            title=title,
            items=items,
            rows=1,
            cols=5,
            filename=filename,
            figsize=(18, 4)
        )

    def show_final_comparison(self, images, filter_processor, morphology_processor):
        rows = len(images)

```

```

cols = 6

plt.figure(figsize=(18, 5 * rows))
plt.suptitle("Final preprocessing comparison", fontsize=16)

for row_index, image_item in enumerate(images):
    original = image_item.bgr
    grayscale = self.color_processor.to_grayscale(original)
    gaussian = filter_processor.gaussian_blur(original, 11)
    median = filter_processor.median_blur(original, 11)
    sharpened = filter_processor.sharpen(original)
    binary = morphology_processor.binarize(original)
    morph = morphology_processor.opening(binary, 5)

    row_items = [
        ("Original", original),
        ("Grayscale", grayscale),
        ("GaussianBlur", gaussian),
        ("MedianBlur", median),
        ("Sharpening", sharpened),
        ("Morphology", morph)
    ]

    for col_index, (title, image) in enumerate(row_items):
        prepared_image, cmap = self.prepare_image(image)
        plot_index = row_index * cols + col_index + 1

        plt.subplot(rows, cols, plot_index)
        plt.imshow(prepared_image, cmap=cmap)
        plt.title(f"{image_item.name}\n{title}")
        plt.axis("off")

plt.tight_layout()

output_path = os.path.join(self.output_dir, "07_final_comparison.png")
plt.savefig(output_path, dpi=200, bbox_inches="tight")
plt.show()

class PreprocessingExperiment:
    def __init__(self, image_paths):
        self.repository = ImageRepository(image_paths)
        self.color_processor = ColorSpaceProcessor()
        self.filter_processor = FilterProcessor()
        self.noise_processor = NoiseProcessor()
        self.morphology_processor = MorphologyProcessor(self.color_processor)
        self.visualizer = Visualizer(self.color_processor)

    def run(self):
        images = self.repository.load()

        self.visualizer.show_originals(images)

        for image_item in images:
            self.visualizer.show_color_spaces(image_item)

        selected_image = images[0]
        text_image = images[2]

        smoothing_results = self.filter_processor.get_smoothing_results(
            selected_image.bgr,
            [5, 11, 21]
        )

        self.visualizer.show_smoothing(selected_image, smoothing_results)

```

```

sharpened = self.filter_processor.sharpen(selected_image.bgr)
self.visualizer.show_sharpening(selected_image, sharpened)

morphology_results = self.morphology_processor.get_morphology_results(
    text_image.bgr,
    [3, 5, 9]
)

self.visualizer.show_morphology(text_image, morphology_results)

gaussian_noisy = self.noise_processor.add_gaussian_noise(selected_image.bgr)
salt_pepper_noisy = self.noise_processor.add_salt_pepper_noise(selected_image.bgr)

gaussian_filtered = [
    ("GaussianBlur", self.filter_processor.gaussian_blur(gaussian_noisy, 5)),
    ("MedianBlur", self.filter_processor.median_blur(gaussian_noisy, 5)),
    ("AverageBlur", self.filter_processor.average_blur(gaussian_noisy, 5))
]

salt_pepper_filtered = [
    ("GaussianBlur", self.filter_processor.gaussian_blur(salt_pepper_noisy, 5)),
    ("MedianBlur", self.filter_processor.median_blur(salt_pepper_noisy, 5)),
    ("AverageBlur", self.filter_processor.average_blur(salt_pepper_noisy, 5))
]

self.visualizer.show_noise_reduction(
    title="Gaussian noise reduction",
    original=selected_image.bgr,
    noisy=gaussian_noisy,
    filtered_results=gaussian_filtered,
    filename="06_gaussian_noise_reduction.png"
)

self.visualizer.show_noise_reduction(
    title="Salt and pepper noise reduction",
    original=selected_image.bgr,
    noisy=salt_pepper_noisy,
    filtered_results=salt_pepper_filtered,
    filename="06_salt_pepper_noise_reduction.png"
)

self.visualizer.show_final_comparison(
    images=images,
    filter_processor=self.filter_processor,
    morphology_processor=self.morphology_processor
)

def main():
    image_paths = {
        "Portrait": "images/portrait.jpg",
        "City": "images/city.jpg",
        "Text": "images/text.jpg"
    }

    experiment = PreprocessingExperiment(image_paths)
    experiment.run()

if __name__ == "__main__":
    main()

```